Apollo Pharmacy Automation Testing Project Using Selenium,PyTest and BDD Behave

Submission Details

Student Name	Nandamuri Mohan Arun
Superset ID	5424478
Website URL	https://www.apollopharmacy.in
Application	Apollo Pharmacy Website
Module	Browse medicine according to health condition
Project 1	Apollo_Pharmacy_Pytest_Framework Selenium Pytest Automation Framework
Project 2	Apollo_Pharmacy_BDD_Behave_Framework Selenium Behave BDD Automation Framework

This document describes the automation testing work completed for the Apollo Pharmacy Buy Medicines workflow using Selenium, Pytest, and Behave BDD frameworks.

Project Description

The project automates the Apollo Pharmacy Buy Medicines workflow using Selenium WebDriver with both Pytest and Behave BDD frameworks.

The automation validates whether a user can successfully open the Apollo Pharmacy website, navigate to Buy Medicines, select health categories, apply product filters, add products to cart, update quantity, proceed to checkout, login using mobile number, and validate address selection functionality.

The framework supports both positive and negative test scenarios along with a complete end-to-end workflow.

The project also validates:

- Product page loading
- Product filtering
- Product cart operations
- Quantity increase and decrease
- Invalid category handling
- Invalid page validation
- Checkout workflow
- Manual OTP handling
- Saved address selection

The framework uses Page Object Model architecture with reusable page methods, logging utilities, screenshots, assertions, and Allure reporting integration.

Tools & Technologies Used

Project 1 – Apollo_Pharmacy_Pytest_Framework

- Python 3.x
- Selenium WebDriver
- Pytest
- Allure Pytest
- OpenPyXL
- ChromeDriver
- Page Object Model (POM)
- Logging Framework
- Excel-Driven Test Data

Project 2 – Apollo_Pharmacy_BDD_Behave_Framework

- Python 3.x
- Selenium WebDriver
- Behave BDD
- Gherkin Syntax
- Allure Behave
- OpenPyXL
- ChromeDriver
- VS Code
- Page Object Model (POM)
- Logging Framework
- Excel-Driven Test Data

Key Features

Project 1 – Pytest Framework

- Selenium automation using Pytest
- Page Object Model framework
- Positive, negative, and end-to-end test coverage
- Excel-driven test data management
- Screenshot capture support
- Logging framework integration
- Allure report integration
- Browser automation using ChromeDriver
- Reusable utility methods

Project 2 – Behave BDD Framework

- Selenium automation using Behave BDD
- Gherkin-based feature files
- Reusable step definitions
- Positive, negative, and end-to-end workflows
- Quantity validation
- Product filter validation
- Pagination handling
- Saved address selection
- Allure reporting integration
- Screenshot attachments
- Logging support

Test Strategy

The framework follows UI automation testing using Selenium WebDriver with a Page Object Model architecture.

Reusable actions are implemented inside page classes while step definitions are separated inside Behave step files.

The framework validates:

- Positive test scenarios
- Negative test scenarios
- End-to-end workflows

Assertions are used throughout the framework to validate:

- Page navigation
- Product loading
- Cart validation
- Quantity updates
- Invalid category handling
- Invalid page validation
- Checkout flow

Synchronization is handled using:

- Explicit waits
- Implicit waits
- Selenium Expected Conditions
- Retry-style interactions
- Dynamic locator handling

Logs and screenshots are captured during execution and attached to Allure reports for debugging and reporting purposes.

Positive Test Scenarios

Positive Scenario 1

Verify Product Add To Cart

Steps:

- Launch Apollo Pharmacy website
- Open Buy Medicines
- Open health category
- Add product to cart
- Open cart
- Validate cart page

Positive Scenario 2

Verify Filters And Product Addition

Steps:

- Launch website
- Open category
- Apply filters
- Add product to cart
- Validate product added successfully

Positive Scenario 3

Verify Navigation To Another Page

Steps:

- Launch website
- Open category
- Navigate to another page using pagination

- Add product from next page
- Validate navigation and cart functionality

Positive Scenario 4

Verify Quantity Increase And Decrease

Steps:

- Launch website
- Add product to cart
- Increase quantity
- Decrease quantity
- Validate quantity updates successfully

Negative Test Scenarios

Negative Scenario 1
Verify Invalid Category Handling

Steps:
• Launch website
• Open invalid category index
• Capture exception
• Validate exception handling

Negative Scenario 2
Verify Invalid Page Navigation

Steps:
• Launch website
• Navigate to invalid page URL
• Validate invalid page handling
• Verify page validation using assertions

End-to-End Workflow

The complete end-to-end workflow validates the major Apollo Pharmacy user journey.

Workflow Steps:

- Launch Apollo Pharmacy website
- Click Buy Medicines
- Open category dynamically
- Apply filters
- Add product to cart
- Open cart
- Proceed to checkout
- Enter mobile number dynamically
- Wait for manual OTP entry
- Open saved address section
- Select Office saved address
- Validate address selection
- Validate checkout workflow

The end-to-end scenario validates complete integration between multiple modules and verifies that the user workflow functions correctly.

Execution Logs

2026-05-23 12:21:47 - INFO - Homepage loaded

2026-05-23 12:22:00 - INFO - Buy Medicines clicked

2026-05-23 12:22:20 - INFO - Category opened

2026-05-23 12:22:43 - INFO - Filters applied

2026-05-23 12:23:03 - INFO - Product added to cart

2026-05-23 12:23:13 - INFO - Cart validation passed

2026-05-23 12:23:25 - INFO - Proceed clicked

2026-05-23 12:23:44 - INFO - Mobile entered

2026-05-23 12:24:58 - INFO - Address validation completed

Log File Location:

logs/test_execution.log

Allure Reports

The framework uses Allure Reports for execution reporting and screenshot attachments.

The reports contain:
• Test execution summary
• Passed and failed scenarios
• Step execution details
• Screenshots
• Logs
• Execution timing
• Assertions and validations

Report Locations:

Project 1:
reports/allure-report/index.html

Project 2:
reports/allure-report/index.html

The generated reports validate successful execution of:
• Positive scenarios
• Negative scenarios
• End-to-end workflow
• Quantity validation

Project Structure

Apollo_Pharmacy_Pytest_Framework/

|-- config/

|-- pages/

|-- tests/

|-- reports/

|-- screenshots/

|-- logs/

|-- utils/

|-- test_data/

|-- requirements.txt

|-- pytest.ini

Apollo_Pharmacy_BDD_Behave_Framework/

|-- config/

|-- features/

| |-- positive.feature

| |-- negative.feature

| |-- end_to_end.feature

| |-- steps/

|

|-- pages/

|-- reports/

|-- screenshots/

|-- logs/

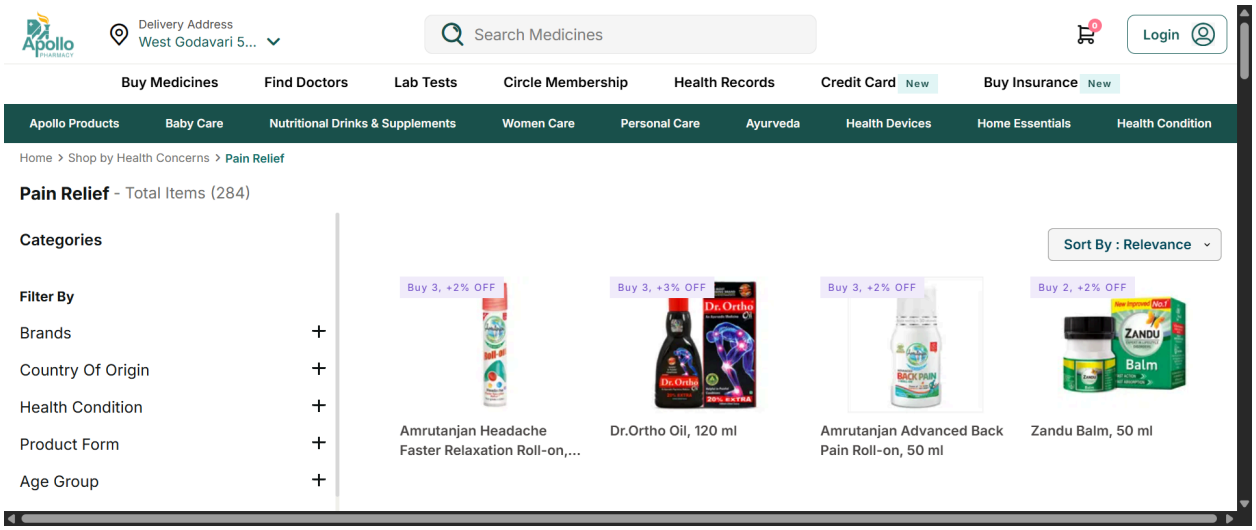
|-- utils/

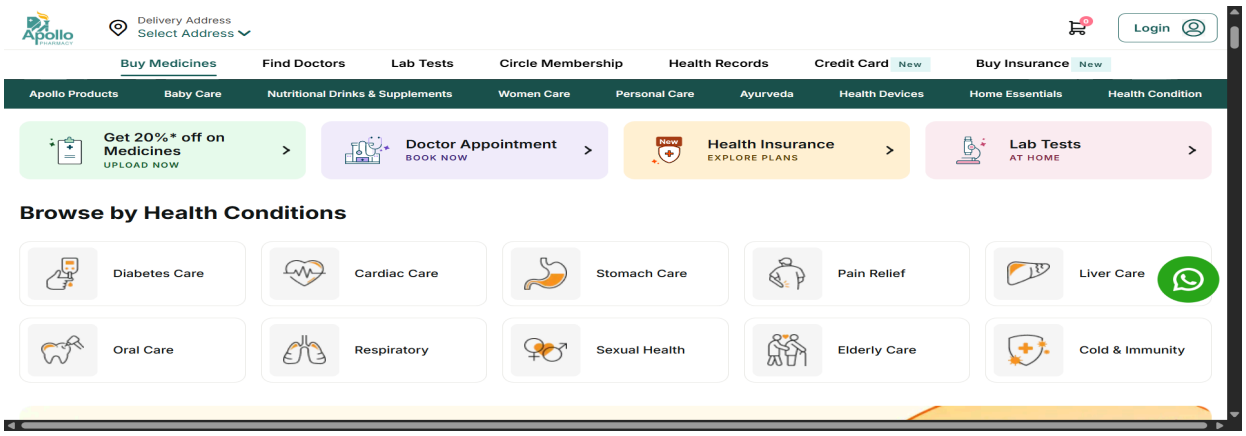
|-- test_data/

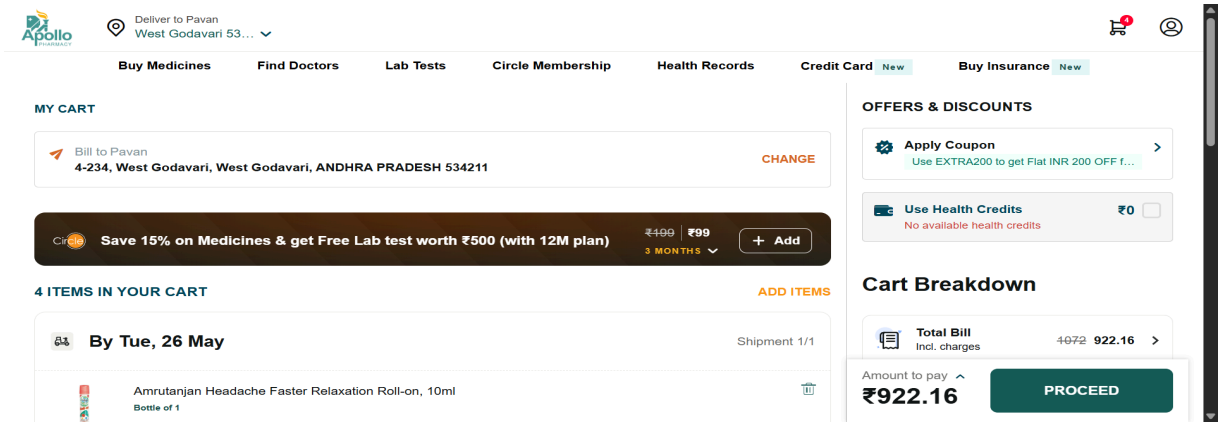
|-- requirements.txt

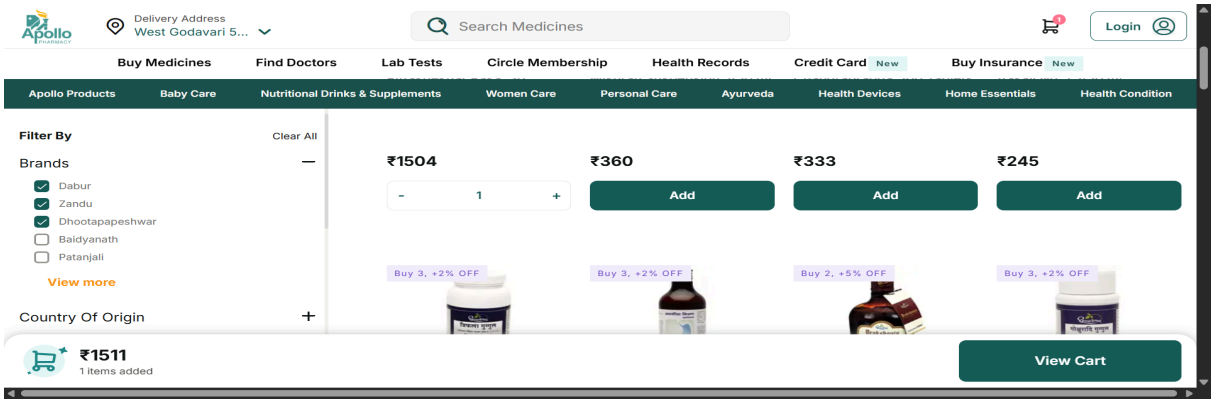
|-- behave.ini

Screenshots:









Framework Components

- Feature Files
- Step Definitions
- Page Object Model Classes
- Utility Functions
- Excel Data Handling
- Logging Utilities
- Screenshot Utilities
- Allure Report

Conclusion

The Apollo Pharmacy automation testing project was successfully completed using Selenium with Pytest and Behave BDD frameworks.

The framework validates important user workflows such as:

- Product selection
- Product filtering
- Cart validation
- Quantity updates
- Checkout process
- Login workflow
- Address selection
- Invalid scenario handling

The framework also includes:

- Page Object Model architecture
- Reusable step definitions
- Assertions and validations
- Logging framework
- Screenshot support
- Excel-driven test data
- Allure reporting integration

Overall, the project demonstrates practical UI automation testing for a real-world pharmacy application using maintainable and scalable automation framework design.